

Software Design and Development Project

SWE4099

Submitted BY

17MIS1031 - MALLIKHARJUNA

Guide: Dr.R.Vedhapriyavadhana

Project Title:

SKETCH

IN

AIR

May - 2022

➤ Contents:

1. Project Motive
2. Proposed System
3. Gesture Recognition
4. Architecture
5. Hardware-Software Requirements
6. Architecture Implementation
7. Discussion on results
8. References

1. Project Motive:

The Motive of this project is to make use of advanced **image/video processing techniques** that will enable the user to sketch in air in different colors thereby eliminating the need to use a mouse or a drawing graphics tablet.

How can we achieve it with OpenCV.....?

2. Proposed System

Sketch In Air is a hands-free digital drawing that utilizes a OpenCV , TensorFlow to recognize and map hand gestures onto screen. The user's "brush" can be modified in size and color by using built-in buttons.

The direction of the brush is controlled completely using open source OpenCV software and modified to map the pointer finger onto the screen using **MASK** following a calibration screen to measure and record the color of the user's hand/object tip.

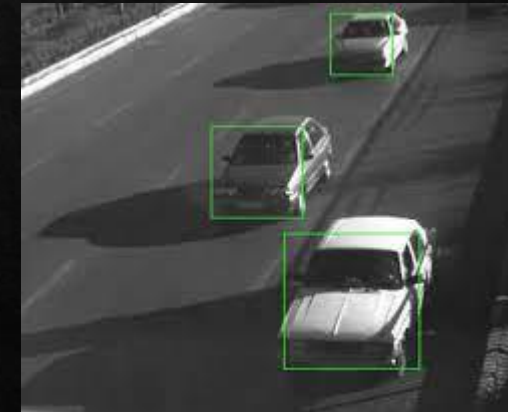
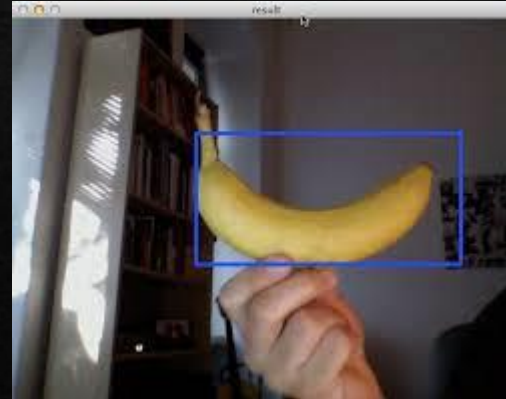
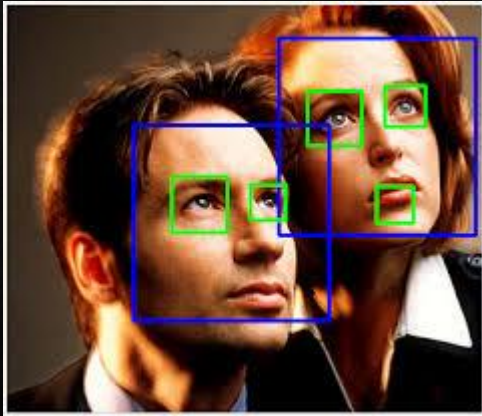
3. Gesture Recognition:

Object Detection : Static

What exactly is object detection.

-Detecting the Objects that we want to identify from a picture or video(sequence of Pictures)

Formal Way :- Detecting a particular class of instances in digital images and videos



➤ Gesture Recognition

One that detects a hand in the region of interaction and performs user classification, and another that performs gesture recognition.

The authors have also demonstrated the feasibility of the system by using a challenging RGBD hand gesture data set collected under settings of common illumination **variation and occlusion**.

➤ Gesture Recognition

I began our project by searching for open source hand gesture recognition software that utilized OpenCV in combination with Python.

The OpenCv authors have developed a vision-based system that employs a combined RGB and depth descriptor to classify hand gestures.

Color detection is an image processing technique where we can detect any color in a given range of **HSV** color space.

➤ Gesture Recognition

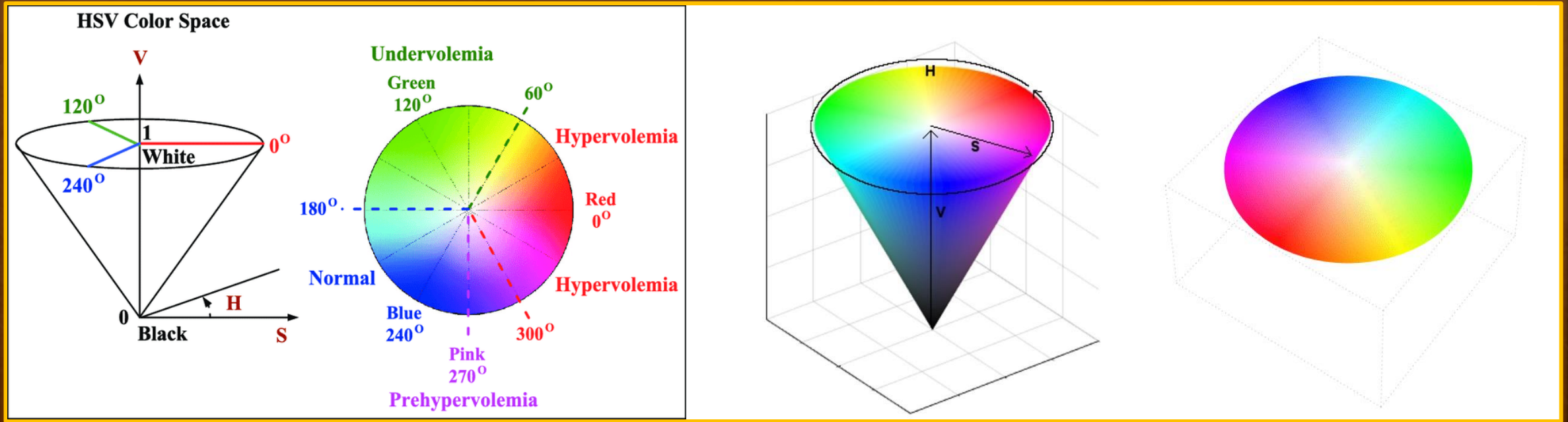
OpenCV is an open-source computer vision library for performing various advanced image processing tasks.

We'll use color detection and segmentation techniques to achieve this objective.

Here, we're going to use a **green/blue** object to simulate a pencil for the Sketch

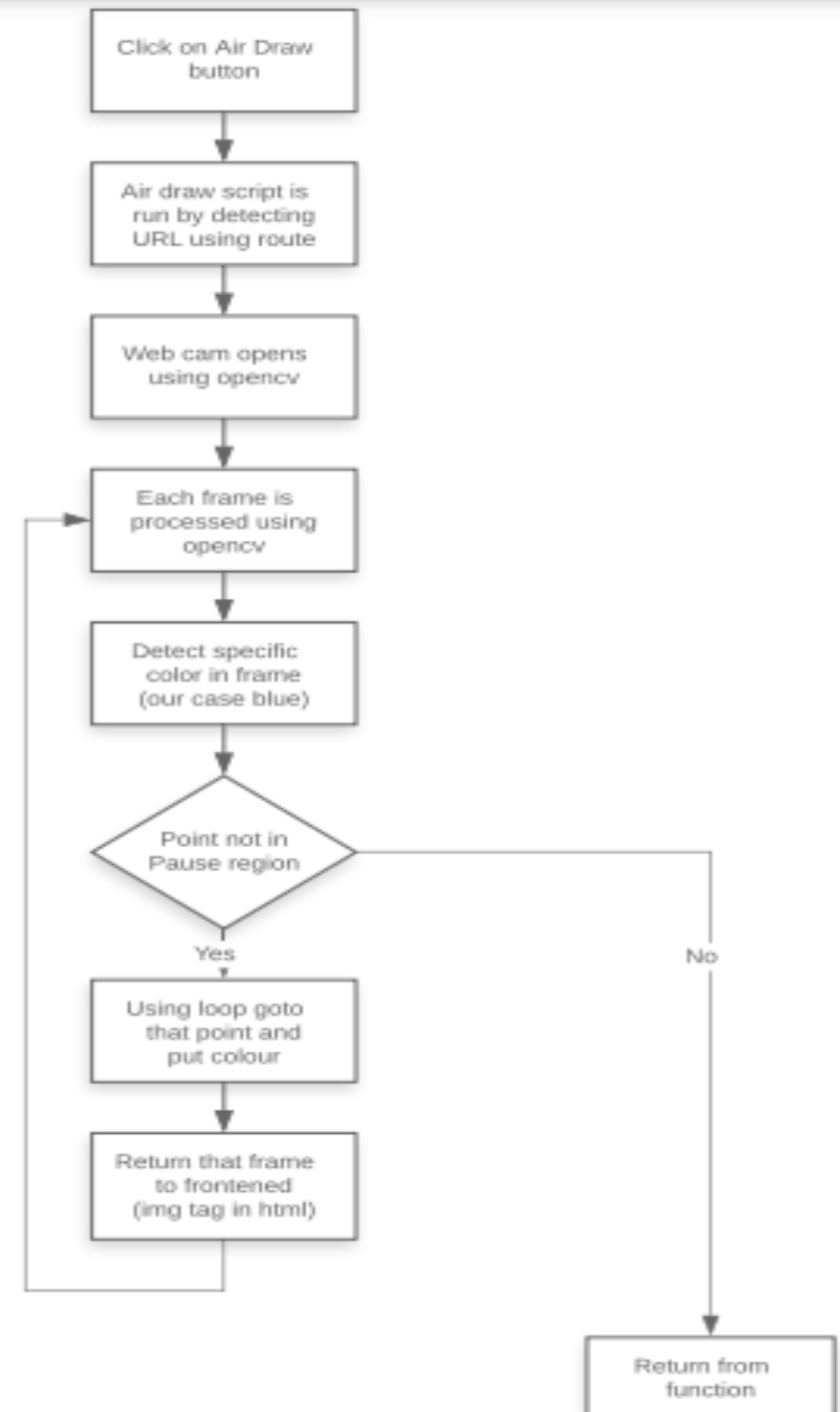
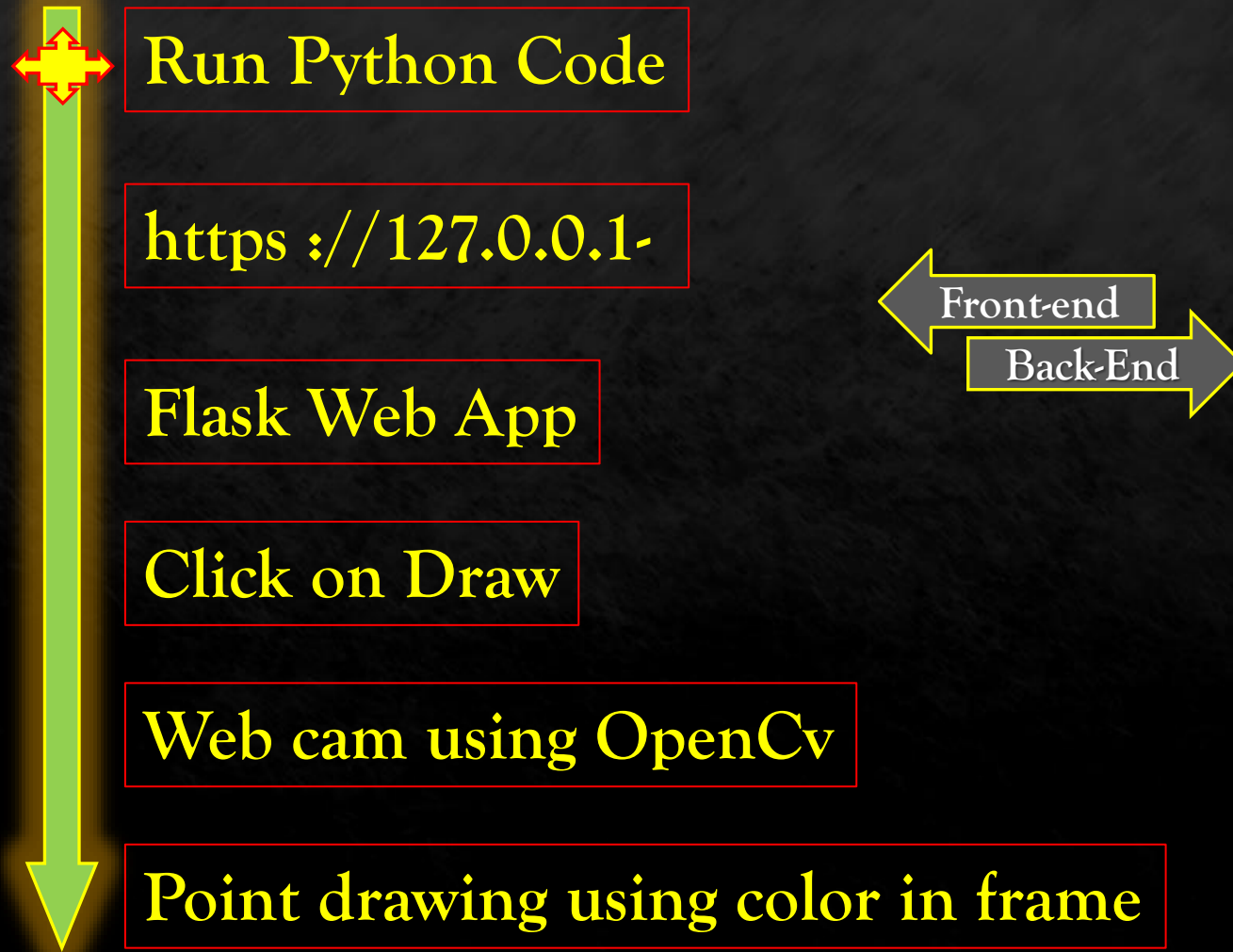
But what is HSV color-space?

HSV stands for HUE, SATURATION, and VALUE (or brightness). It is basically a cylindrical color space.

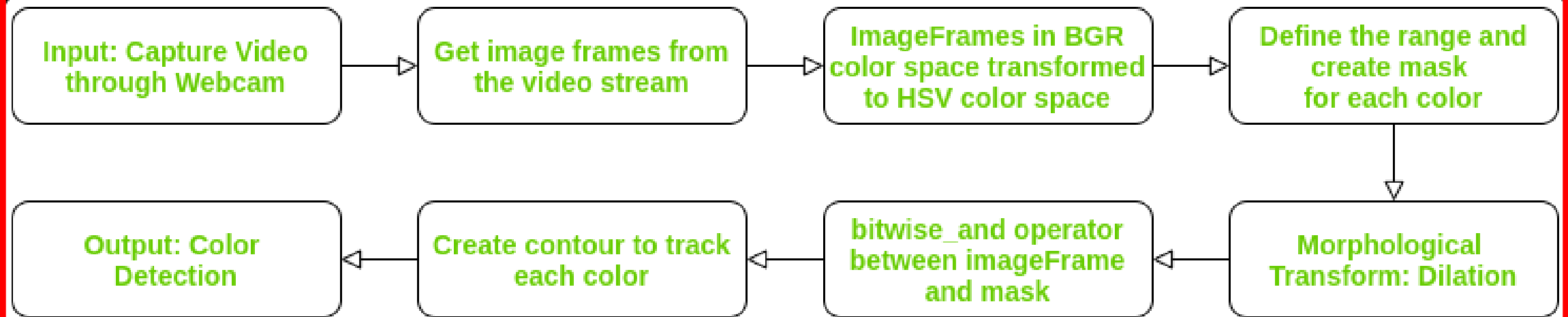


HUE – The hue encodes color information in angular dimension.
SATURATION – Saturation encodes the intensity of color.
VALUE – Value represents the brightness of the color.

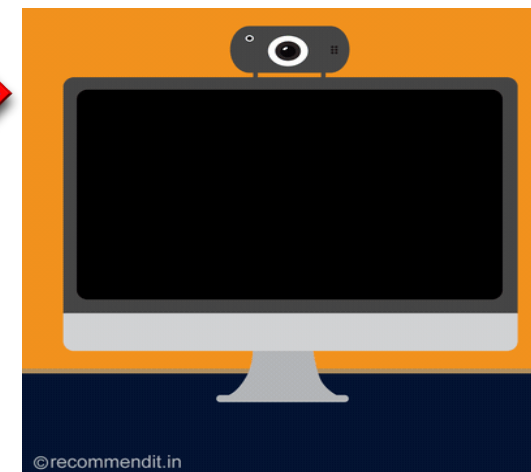
4. Architecture



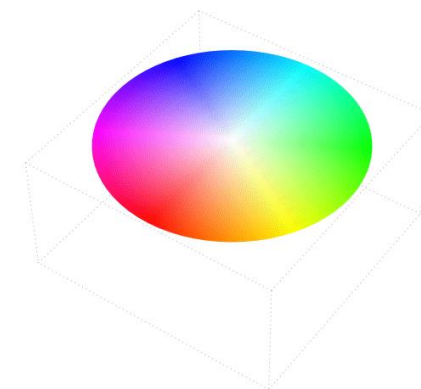
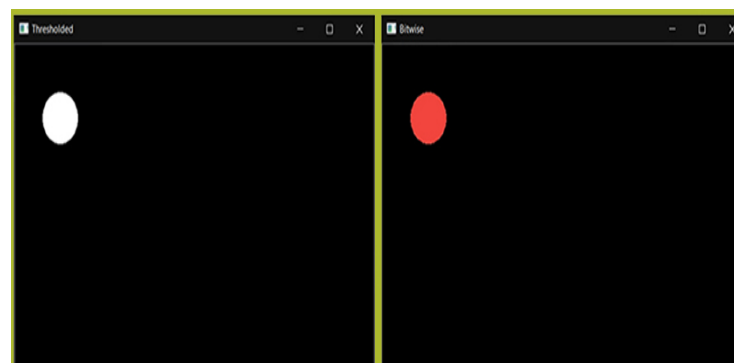
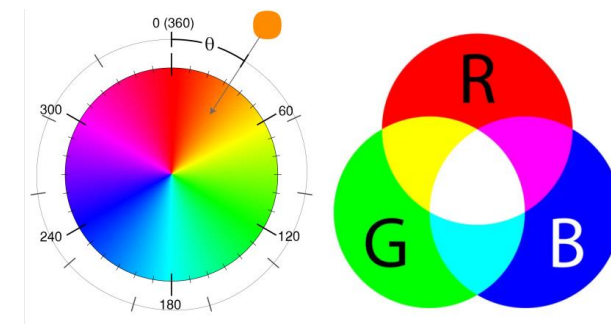
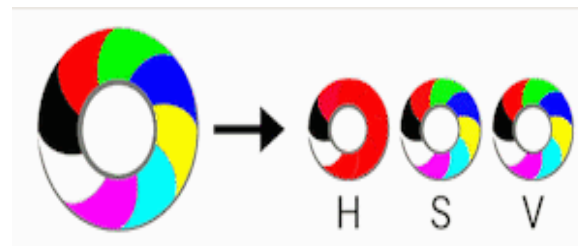
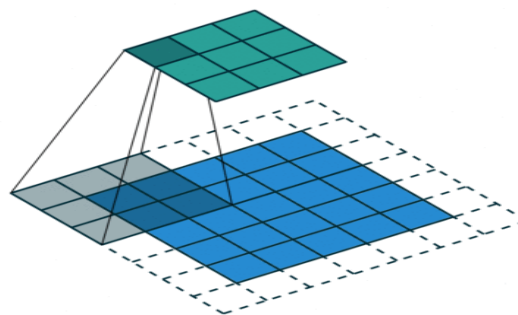
4. Architecture



Work Flow



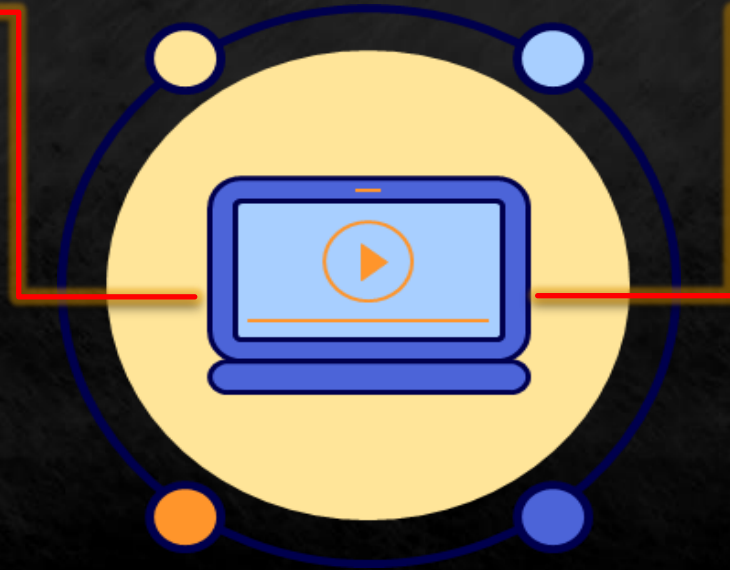
Flask



➤ REQUIREMENTS:

SOFTWARE

- ❖ Python v3.10.1
- ❖ Flask
- ❖ openCV-cv2
- ❖ Tensorflow-latest
- ❖ Keras
- ❖ HTML
- ❖ Bootstrap



HARDWARE

- ❖ NVIDIA graphics card & respective drivers
- ❖ CUDA driver v11.1 (for tensorflow)
- ❖ cuDNN v8.1
- ❖ Inbuild webcam
- ❖ Ram-8gb
- ❖ 2gb-disk-space
- ❖ Any color monitor

4. Implementation:

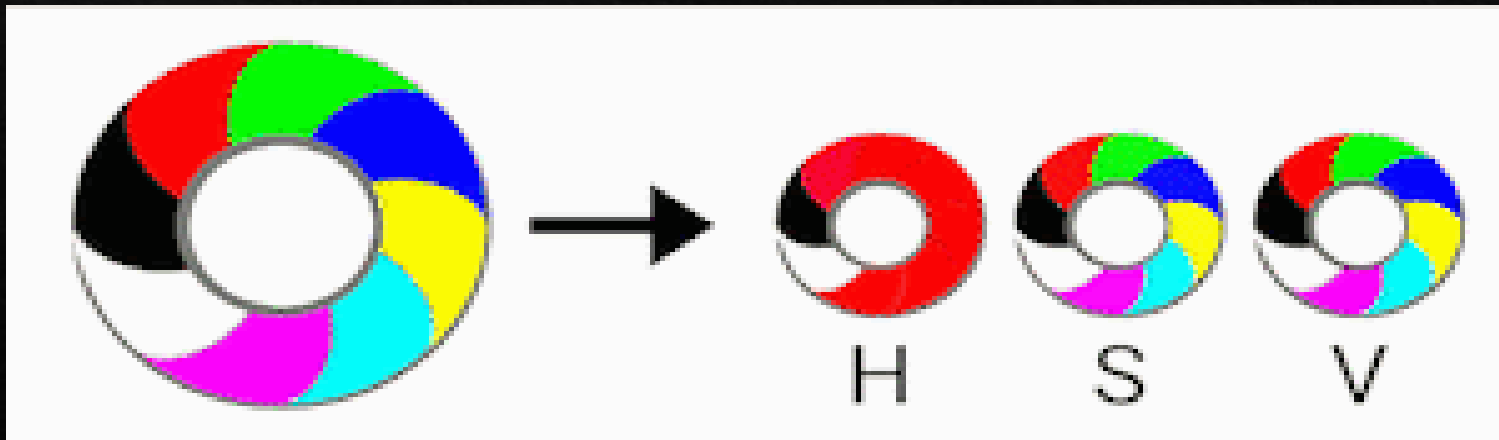
Import necessary **packages** and pre-define some settings:

```
import cv2
from keras.models import load_model
import numpy as np
import os
from collections import deque
from flask import Flask, render_template, Response, redirect, url_for
import time
```

Step1: *Find the color range of the target object and save it.*

• **cap.read()** function reads each frame from the webcam.  Opens Web Cam

- > Here we're using a BLUE object to simulate the pencil of our canvas.
- > OpenCV reads frames as BGR color space.
- > But to detect colors, we have to convert the frame to the HSV color space.



Convert the frame BGR to HSV color space

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

Create a binary segmented mask of any color

```
mask = cv2.inRange(hsv, lower_bound, upper_bound)
```

Add some dialation to increase segmented area

```
mask = cv2.dilate(mask, kernel, iterations=1)
```

Color range for detecting Blue color

```
lower_bound = np.array([50,100,100])
```

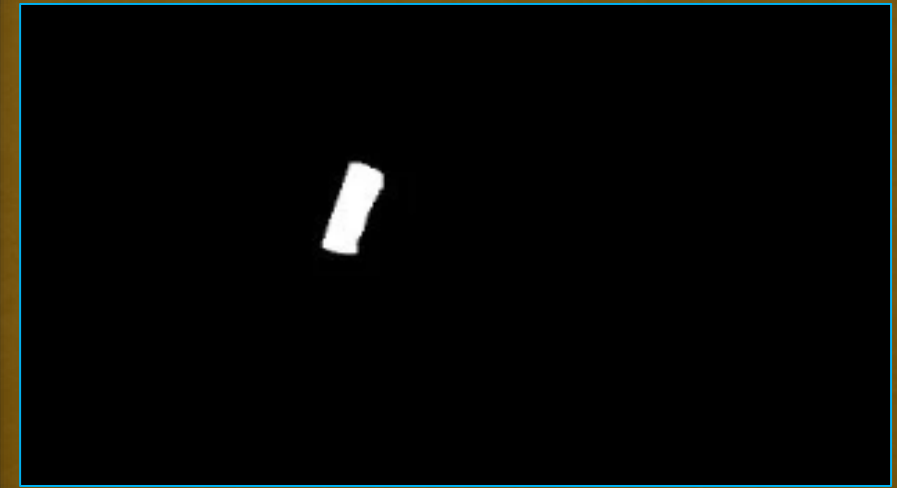
```
upper_bound = np.array([90,255,255])
```

Blue In our case



Set the lower and upper HSV range according Mask .

```
res = cv2.bitwise_and(frame, frame, mask=mask)
```



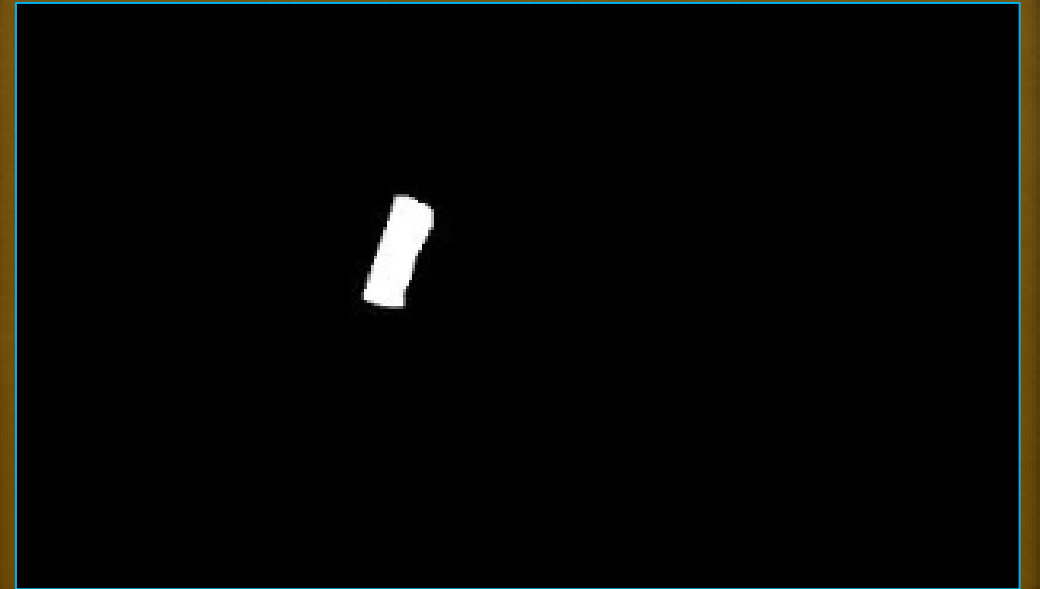
stack the mask, original frame and the filtered result

```
stacked = np.hstack((mask_3, frame, res))
```



Step 2:

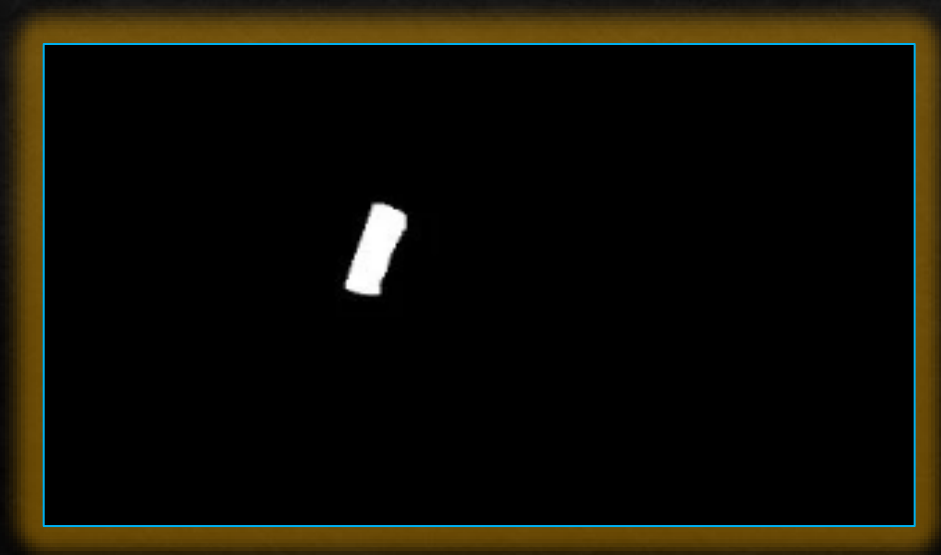
Apply the correct morphological operations to reduce noise in the video



Step 3:

Detect and track the colored object with contour detection.

Now that we have got a **decent mask**, we can use it to detect our pen using **contour detection**. We will draw a bounding box around our object to make sure it's being detected all over the screen.



Step 4:

Find the object's x, y location coordinates to draw on the screen.

#if counterArea $\geq$ countersArea , make it as New Pointer

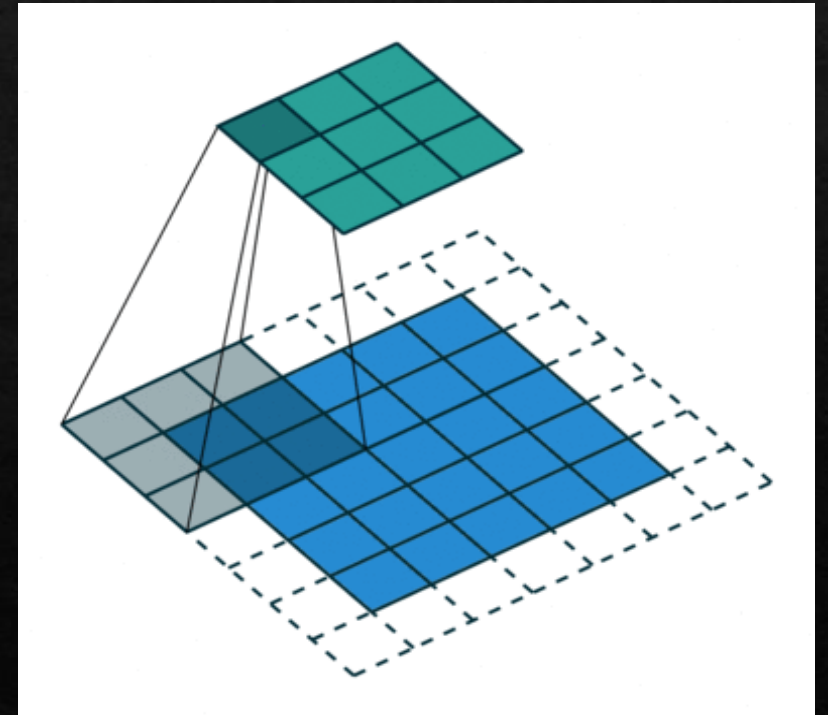
```
c = max(contours, key = cv2.contourArea)
```

update pointer for every frame using BoundingRect

```
x2,y2,w,h = cv2.boundingRect(c)
```

After the line is drawn the new points become the previous points.

```
line= cv2.line(mask, (x1,y1),(x2,y2), [255,0,0])
```



Step 5 :

Add a Erase & Wiper functionality to wipe off the whole screen

We make the mask_position as true where x,y co-ordinates are true and make the same x,y location frame as false

Eraser:

```
mask_position = cv2.bitwise_not(mask))  
    if frame[0: 50, 0: 50] :  
        frame=flase
```

Simply make the line[] array empty

```
line= cv2.line(mask, (x1,y1),(x2,y2), [255,0,0])
```

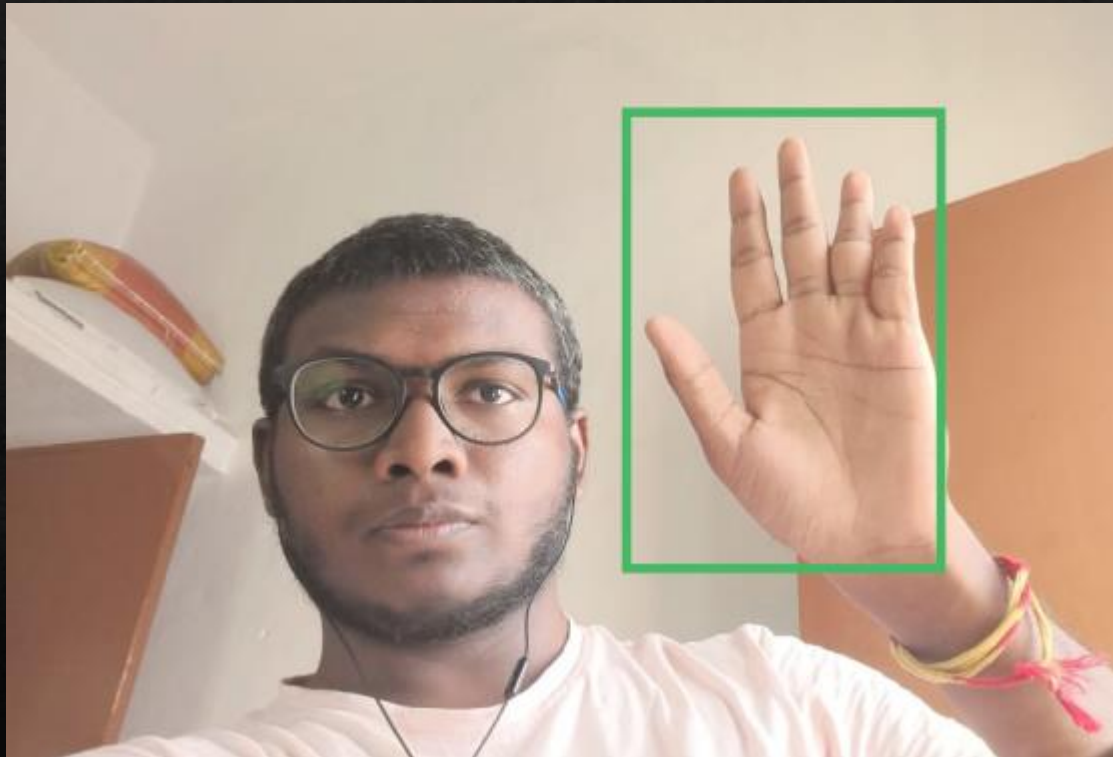
Clear All:



```
line = [ ]
```

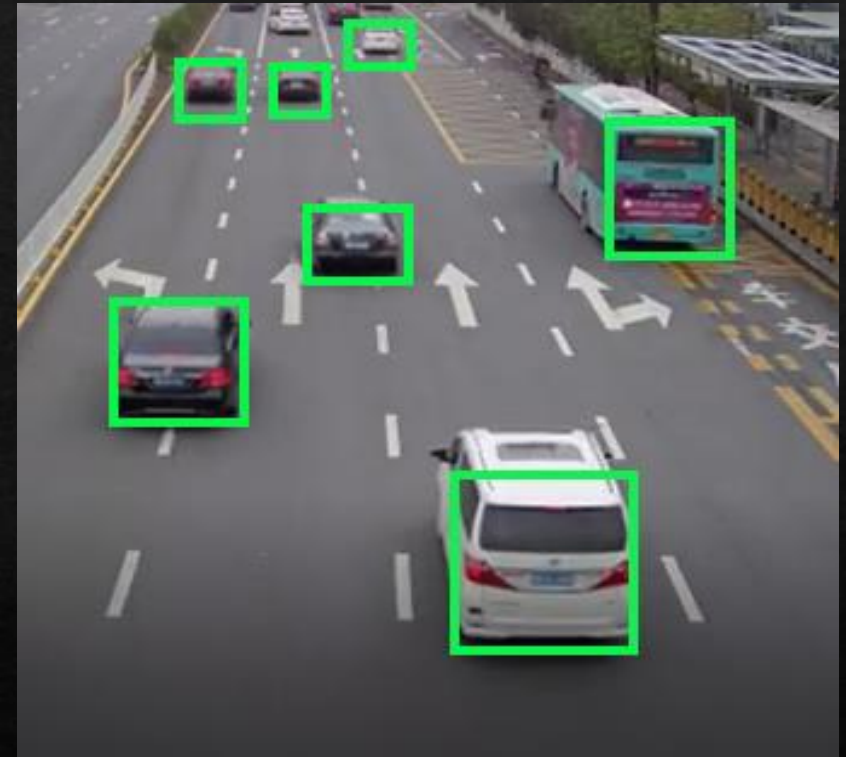
Checking OpenCv Packages:

Hand Detection



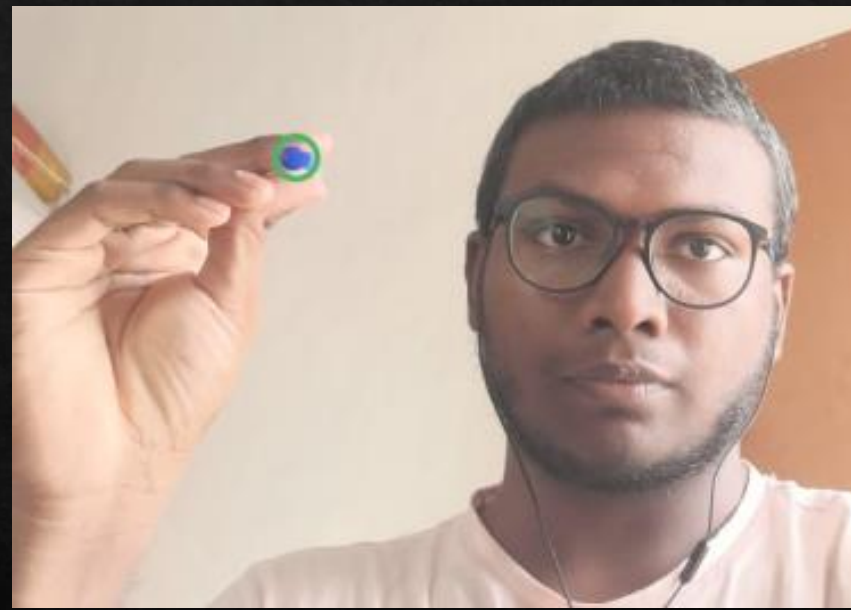
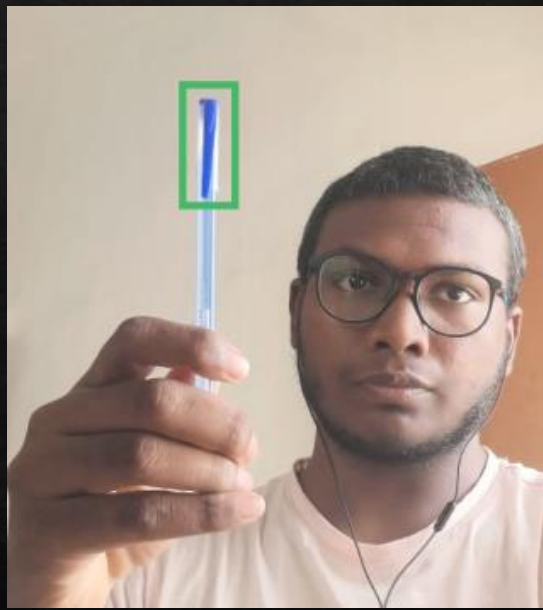
Real Time-dynamic

Object-Detection

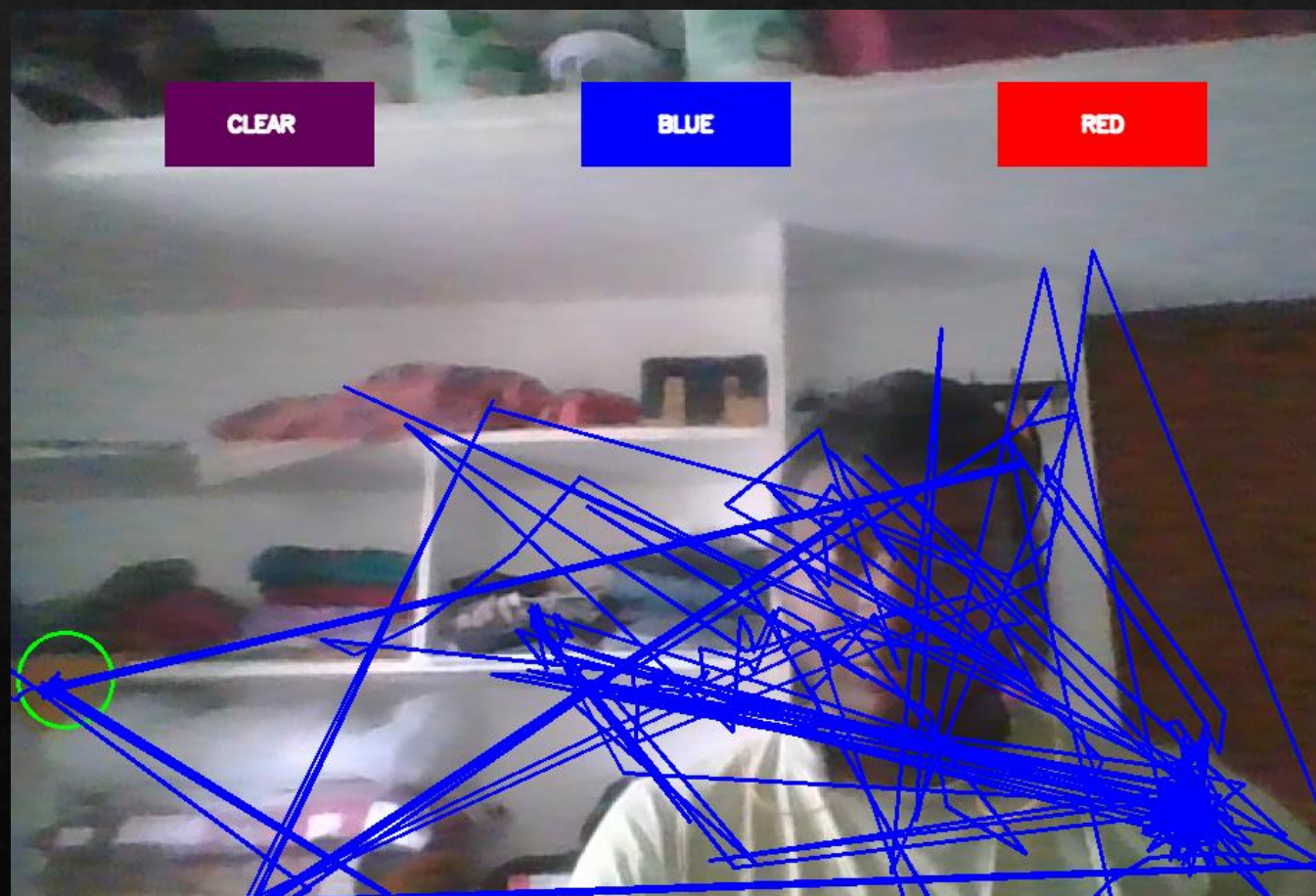


Static

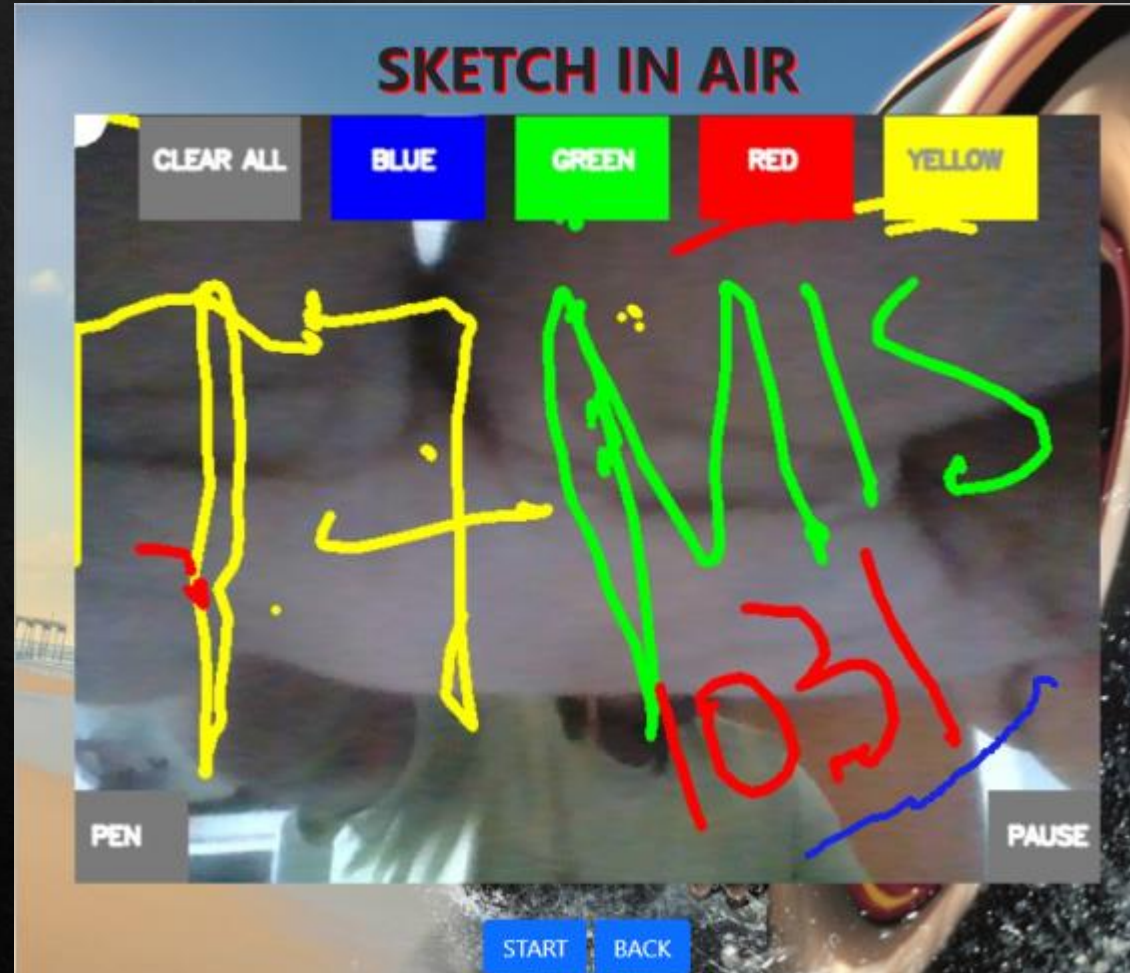
Colour Detection



Initial-App-without-TensorFlow

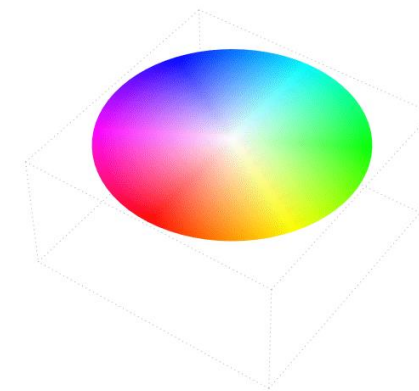
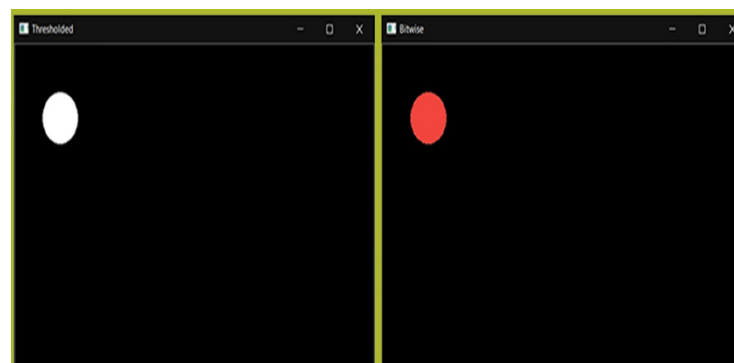
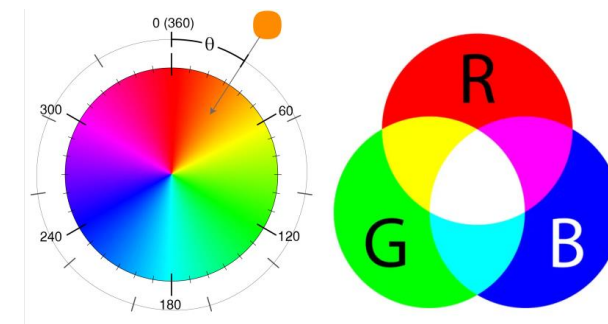
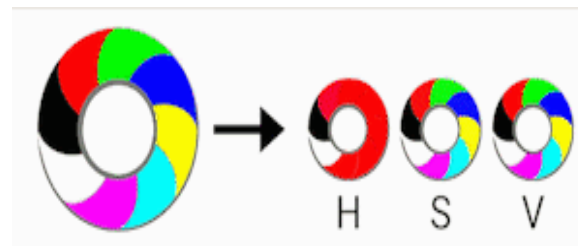
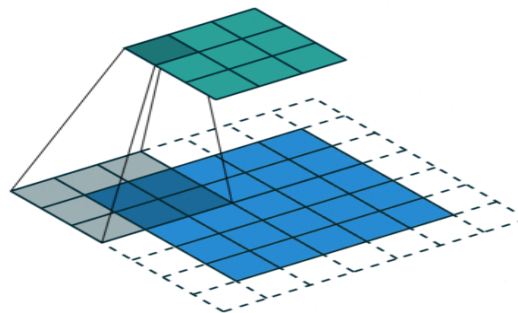


App-with-TensorFlow





Flask



THANK YOU

References:

<https://stackabuse.com/object-detection-with-imageai-in-python/>

<https://etrr.springeropen.com/articles/10.1186/s12544-019-0390-4>

<https://towardsdatascience.com/object-detection-with-tensorflow-model-and-opencv-d839f3e42849>

<https://www.pyimagesearch.com/2018/11/12/yolo-object-detection-with-opencv/>

<https://learncodebygaming.com/blog/training-a-cascade-classifier>

https://www.bogotobogo.com/python/OpenCV_Python/python_opencv3_Image_Object_Detection_Face_Detection_Haar_Cascade_Classifiers.php

<https://www.sciencedirect.com/topics/computer-science/gesture-recognition>

<https://data-flair.training/blogs/project-in-python-colour-detection/>

<https://pysource.com/2021/10/19/simple-color-recognition-with-opencv-and-python/>

<https://www.apativ.com/en/insights/article/what-is-gesture-recognition>

